

```

import numpy as np
import cv2
import imageio.v3 as iio
import matplotlib.pyplot as plt

INPUT_VIDEO = "ar5.gif"
OUTPUT_VIDEO = "a5_out.mp4"

N_POINTS = 30
PATCH = 12
FPS = 30

# =====
# 1.3 Calibrating the Camera
# - Estimate camera projection matrix P (3x4)
# - Use DLT with 2D-3D correspondences
# =====

def estimate_P(Xw, uv):
    N = Xw.shape[0]
    Xh = np.hstack([Xw, np.ones((N, 1))])

    A = np.zeros((2 * N, 12), dtype=float)
    for i in range(N):
        X = Xh[i]
        u, v = uv[i]
        A[2*i, 0:4] = X
        A[2*i, 8:12] = -u * X
        A[2*i+1, 4:8] = X
        A[2*i+1, 8:12] = -v * X

    _, _, VT = np.linalg.svd(A)
    P = VT[-1].reshape(3, 4)
    return P

def project(P, Xw):
    Xh = np.hstack([Xw, np.ones((Xw.shape[0], 1))])
    x = (P @ Xh.T).T
    return np.stack([x[:,0]/x[:,2], x[:,1]/x[:,2]], axis=1)

```

```

# =====
# 1.2 Propagating Keypoints to Other Frames
# - Initialize OpenCV trackers for each keypoint
# - Track keypoints frame-by-frame
# =====

def make_tracker():
    if hasattr(cv2, "legacy") and hasattr(cv2.legacy, "TrackerMedianFlow_create"):
        return cv2.legacy.TrackerMedianFlow_create()
    if hasattr(cv2, "TrackerMedianFlow_create"):
        return cv2.TrackerMedianFlow_create()

    if hasattr(cv2, "legacy") and hasattr(cv2.legacy, "TrackerCSRT_create"):
        return cv2.legacy.TrackerCSRT_create()
    if hasattr(cv2, "TrackerCSRT_create"):
        return cv2.TrackerCSRT_create()

    if hasattr(cv2, "legacy") and hasattr(cv2.legacy, "TrackerKCF_create"):
        return cv2.legacy.TrackerKCF_create()
    if hasattr(cv2, "TrackerKCF_create"):
        return cv2.TrackerKCF_create()

    raise AttributeError("No supported OpenCV tracker found.")

def init_trackers(frame_bgr, uv0, patch=8):
    half = patch // 2
    trackers = []
    for (u, v) in uv0:
        tr = make_tracker()
        bbox = (float(u-half), float(v-half), float(patch), float(patch))
        tr.init(frame_bgr, bbox)
        trackers.append(tr)
    return trackers

def track(trackers, frame_bgr):
    uv = []
    for tr in trackers:
        ok, (x, y, w, h) = tr.update(frame_bgr)
        if not ok:
            uv.append([np.nan, np.nan])

```

```

        else:
            uv.append([x + w/2, y + h/2])
    return np.asarray(uv, dtype=float)

# =====
# 1.4 Projecting a Cube into the Scene
# - Define cube in world coordinates
# - Project using calibrated P
# - Render edges on image
# =====

EDGES = [(0,1),(1,2),(2,3),(3,0),
          (4,5),(5,6),(6,7),(7,4),
          (0,4),(1,5),(2,6),(3,7)]

def cube_vertices(x0, y0, z0, s):
    return np.array([
        [x0, y0, z0],
        [x0+s, y0, z0],
        [x0+s, y0, z0+s],
        [x0, y0, z0+s],
        [x0, y0+s, z0],
        [x0+s, y0+s, z0],
        [x0+s, y0+s, z0+s],
        [x0, y0+s, z0+s],
    ], dtype=float)

def draw_cube(frame_bgr, P, Vw, color=(0,0,255), thickness=2):
    uv = project(P, Vw)

    if not np.isfinite(uv).all():
        return

    uv = uv.astype(np.int32)
    for a, b in EDGES:
        cv2.line(frame_bgr,
                  (int(uv[a,0]), int(uv[a,1])),
                  (int(uv[b,0]), int(uv[b,1])),
                  color, thickness)

```

```

# ===== main =====

frames = list(iio.imiter(INPUT_VIDEO))
if len(frames) == 0:
    raise RuntimeError("No frames read. Check INPUT_VIDEO.")

frame0_rgb = frames[0]
frame0_bgr = cv2.cvtColor(frame0_rgb, cv2.COLOR_RGB2BGR)

# =====
# 1.1 Keypoints with Known 3D World Coordinates
# - Manually mark 2D keypoints in first frame
# - Assign structured 3D world coordinates
# =====

plt.figure()
plt.imshow(frame0_rgb)
plt.title(f"Click {N_POINTS} points in THIS EXACT ORDER:\n"
          f"(1) Top face 4x5 = 20 points (front->back, left->right)\n"
          f"(2) Front face MID line = 5 points\n"
          f"(3) Front face BOTTOM line = 5 points\n"
          f"Then close window.")
uv0 = np.array(plt.ginput(n=N_POINTS, timeout=0), dtype=float)
plt.close()

H = 2.0
NX, NZ = 5, 4

Xw = []

for z in range(NZ):
    for x in range(NX):
        Xw.append([x, H, z])

for x in range(NX):
    Xw.append([x, H/2, 0])

for x in range(NX):
    Xw.append([x, 0, 0])

```

```

Xw = np.array(Xw, dtype=float)

# =====
# 1.2 Initialize trackers
# =====

trackers = init_trackers(frame0_bgr, uv0, patch=PATCH)

# =====
# 1.4 Define cube placement
# =====

x0, y0, z0, s = 1.5, H, 1.0, 1.0
Vw = cube_vertices(x0, y0, z0, s)

# =====
# 1.3 Frame-by-frame calibration and projection
# =====

out = []
uv_prev = uv0.copy()
P_prev = None

for fr_rgb in frames:
    fr_bgr = cv2.cvtColor(fr_rgb, cv2.COLOR_RGB2BGR)

    uv = track(trackers, fr_bgr)

    bad = np.isnan(uv[:,0]) | np.isnan(uv[:,1])
    uv[bad] = uv_prev[bad]
    uv_prev = uv

    P = estimate_P(Xw, uv)

    uv_hat = project(P, Xw)
    err = np.nanmean(np.linalg.norm(uv_hat - uv, axis=1))

    if (P_prev is not None) and (not np.isfinite(err) or err > 10.0):
        P = P_prev

```

```
    else:
        P_prev = P

    draw_cube(fr_bgr, P, Vw)
    out.append(cv2.cvtColor(fr_bgr, cv2.COLOR_BGR2RGB))

iio.imwrite(OUTPUT_VIDEO, np.stack(out), fps=FPS)
print("Wrote:", OUTPUT_VIDEO)
```